



Lista 5 - Estrutura de Repetição

Exercícios de Revisão

1) Desenvolva um programa que solicite ao usuário dois valores positivos, correspondentes ao **limite inferior** e ao **limite superior** de um intervalo. O programa deve:

1. Validar a entrada, garantindo que ambos os valores sejam positivos e que o limite superior seja maior ou igual ao limite inferior;
2. Identificar e exibir todos os números primos pertencentes ao intervalo, em **ordem crescente**;
3. Para cada número primo encontrado, calcular e apresentar também a **soma de seus dígitos**.

Exemplo de entrada e saída:

```
Digite um valor positivo para o limite inferior do intervalo: 100
Digite um valor positivo para o limite superior do intervalo: 150
Primo 101 => Soma dos digitos = 2
Primo 103 => Soma dos digitos = 4
Primo 107 => Soma dos digitos = 8
Primo 109 => Soma dos digitos = 10
Primo 113 => Soma dos digitos = 5
Primo 127 => Soma dos digitos = 10
Primo 131 => Soma dos digitos = 5
Primo 137 => Soma dos digitos = 11
Primo 139 => Soma dos digitos = 13
Primo 149 => Soma dos digitos = 14
```

2) Escreva um programa em linguagem C que leia caracteres um por vez, até que seja digitado o caractere '0'. Esse caractere atua como sinal de parada e não deve ser considerado no processamento. Para cada caractere lido (exceto '0'):

- Converta-o para seu valor numérico correspondente na tabela ASCII.
- Verifique se esse número é par.
- Exiba o caractere, seu valor ASCII e se ele é par ou não.

Ao final, o programa deve exibir a quantidade total de caracteres cujo valor ASCII é par, desconsiderando o caractere de parada '0'.

Exemplo de entrada e saída:

Digite caracteres (digite '0' para parar):

```
A
Caractere: 'A' | ASCII: 65 => impar
B
Caractere: 'B' | ASCII: 66 => par
c
Caractere: 'c' | ASCII: 99 => impar
D
Caractere: 'D' | ASCII: 68 => par
h
Caractere: 'h' | ASCII: 104 => par
j
Caractere: 'j' | ASCII: 106 => par
C
Caractere: 'C' | ASCII: 67 => impar
0
```

Total de caracteres com valor ASCII par: 4

3) Desenvolva um programa que solicite ao usuário um número inteiro positivo, denominado **num**. O programa deve:

1. Validar a entrada, garantindo que o valor informado seja **positivo**;
2. Identificar e exibir todos os **números primos** no intervalo que vai de **num** até **num + 10** (inclusive);
3. Calcular e apresentar a **quantidade total** de números primos encontrados nesse intervalo.

Exemplo de entrada e saída:

```
Informe um numero positivo: 10
10 ==> 2 3 5 7 ==> 4 primo(s).
11 ==> 2 3 5 7 11 ==> 5 primo(s).
12 ==> 2 3 5 7 11 ==> 5 primo(s).
13 ==> 2 3 5 7 11 13 ==> 6 primo(s).
14 ==> 2 3 5 7 11 13 ==> 6 primo(s).
15 ==> 2 3 5 7 11 13 ==> 6 primo(s).
16 ==> 2 3 5 7 11 13 ==> 6 primo(s).
17 ==> 2 3 5 7 11 13 17 ==> 7 primo(s).
18 ==> 2 3 5 7 11 13 17 ==> 7 primo(s).
19 ==> 2 3 5 7 11 13 17 19 ==> 8 primo(s).
20 ==> 2 3 5 7 11 13 17 19 ==> 8 primo(s).
```

4) Elabore um programa que apresente todos os números primos existentes em um intervalo definido pelo usuário, sendo os limites **inferior** e **superior** informados na entrada. Os números primos encontrados devem ser exibidos com **n valores por linha**, onde **n** também será informado pelo usuário.

O programa deve realizar as seguintes validações:

- **n** deve ser maior que 0;
- O **limite inferior** deve ser maior que 1;
- O **limite superior** deve ser maior ou igual ao limite inferior.

Exemplo de entrada e saída:

```
Informe o valor do limite inferior do intervalo: 2
Informe o valor do limite superior do intervalo: 100
Informe quantos numeros deseja imprimir por linha: 5
  2   3   5   7  11
13  17  19  23  29
31  37  41  43  47
53  59  61  67  71
73  79  83  89  97
```